

POLICY REVIEWER CHECKLIST

Policy Reviewer Checklist

dealing cluster

CLUSTER dealing cluster

AUDIENCE Anyone shipping a NetworkPolicy, CNP, CCNP, or Kyverno ClusterPolicy

CLASSIFICATION Internal — keep current as the platform evolves

Contents

01	The model you're working inside	3
02	Before you kubectl apply	3
03	After you kubectl apply	7
04	When to escalate	7
05	Links	7

A short checklist for anyone adding or changing a NetworkPolicy, CiliumNetworkPolicy (CNP), CiliumClusterwideNetworkPolicy (CCNP), or Kyverno ClusterPolicy on the `dealing` cluster.

Origin: an in-place CCNP with `endpointSelector: {}` silently broke the Cilium IngressController and several UIs. We added a Kyverno rule to block that one specific footgun. This checklist exists to catch the other 80% that no automation will catch for us.

The model you're working inside

This cluster uses a **4-layer security model**. Know which layer your change belongs to:

```
Layer 1 – Switch ACLs (your network team owns)
          VLAN-to-VLAN segmentation. Cross-VLAN ports allowlisted explicitly.
          → THIS is the network-layer security boundary. The cluster trusts it.

Layer 2 – Cluster-wide CCNPs (platform team owns; in security/network-policies/)
          ONLY 2 additive policies, both with `enableDefaultDeny: false`:
          • platform-baseline-egress – every pod ↔ in-cluster + DNS + apiserver
          • platform-baseline-ingress – IngressController/Prometheus/kube-system → pods
          These DON'T block anything. They grant common platform access.

Layer 3 – Per-app CNPs (app team owns; in their own namespace)
          Optional. Apps add their own CiliumNetworkPolicy when they need
          tighter than baseline. To override, use `enableDefaultDeny.egress: true`.

Layer 4 – App-level TLS / authn / authz (developers own)
          mTLS, JWT, OAuth – defense in depth on top of network policy.
```

Two consequences for any new policy you write:

1. **Don't try to block by CIDR at the cluster level.** Your switch already does VLAN-level segmentation. Cluster CIDR blocks would either duplicate the switch (no value) or contradict it (broken cross-VLAN traffic). We tried `except: [10.0.0.0/8]` on 2026-05-21 and broke legit cross-VLAN egress. Deleted.
2. **The 2 cluster-wide CCNPs are intentionally permissive.** They use `enableDefaultDeny: false` so they grant access without forcing default-deny on every pod. Per-app CNPs with `enableDefaultDeny: true` are how individual apps lock down.

Before you `kubectl apply`

Run through these questions. If any answer is unclear, write the answer into the policy YAML as a comment before applying — that's the documentation the next person needs.

1. Does the policy live in the repo?

If you're applying live via `kubectl` and the manifest isn't in `security/network-policies/`, `security/kyverno-policies/`, or under `terraform/`, **stop and commit it first**. Cluster state must match IaC. The CCNP that broke us was `kubectl`-applied directly.

2. What does this selector actually match?

For NetworkPolicy / CNP / CCNP, the most common mistake is over-broad selection:

Selector	What it actually matches
<code>endpointSelector: {}</code>	EVERY endpoint identity, including reserved (<code>ingress</code> , <code>host</code> , <code>world</code> , <code>kube-apiserver</code> , <code>remote-node</code>). Almost always wrong. Kyverno now blocks this for CCNPs.
<code>podSelector: {}</code> (K8s NetworkPolicy)	Every pod in the namespace. Sometimes correct (e.g. a namespace-wide deny).
<code>endpointSelector: matchExpressions: [{key: k8s.io.kubernetes.pod.namespace, operator: Exists}]</code>	Every real pod cluster-wide, but not reserved identities. Use this when you want "every workload".
<code>endpointSelector: matchLabels: {<specific label>: <value>}</code>	Only pods with that label. Most policies should be this narrow.

For Kyverno ClusterPolicies, check `match` and `exclude` :

- `match: any: - resources: kinds: [Pod]` matches every pod. Almost always you want `match` plus an `exclude` block for `kube-system` , `kyverno` , `cilium-secrets` , and any other system namespace.
- A policy without `exclude` is at risk of breaking platform components.

3. Will this policy force default-deny on any pod?

In Cilium 1.14+, every CNP/CCNP has an implicit `enableDefaultDeny` . The default behavior:

- For a CNP/CCNP with egress rules → `enableDefaultDeny.egress: true` (deny everything except listed)
- For a CNP/CCNP with ingress rules → `enableDefaultDeny.ingress: true`
- A policy with NO rules selects pods but doesn't enforce anything

For the dealing cluster's CCNPs (Layer 2), always set `enableDefaultDeny: false` . We learned this on 2026-05-21 — a CCNP that selects every pod and forces default-deny is the broad sledgehammer that breaks every system pod the platform needs (DNS, ingress controller, metrics-server, hubble-relay, cert-manager → ACME, etc).

```
spec:
  endpointSelector: {...}
  enableDefaultDeny:
    egress: false      # ← critical for cluster-wide CCNPs
    ingress: false     # ← (if it's an ingress policy)
  egress:
    - ...
```

For per-app CNPs (Layer 3), use `enableDefaultDeny: true` deliberately when you want a specific app locked down:

```
# Example: payments app, only stripe.com over TLS allowed
spec:
  endpointSelector:
    matchLabels:
      app: payments
  enableDefaultDeny:
    egress: true      # ← deliberately deny everything else
  egress:
    - toFQDNs: [{matchPattern: "*.stripe.com"}]
      toPorts: [{ports: [{port: "443", protocol: TCP}]}]
```

Mental test: "What's the smallest selector that captures only what I need? Why isn't *that* the selector?"

4. What identities/CIDRs does it allow?

For each rule in `egress` / `ingress` :

Allow shape	Catches
<code>toEntities: [kube-apiserver]</code>	Just the apiserver (CIDRs tagged with the <code>kube-apiserver</code> reserved identity)
<code>toEntities: [host, remote-node]</code>	Node IPs only — does not match pod-CIDR traffic
<code>toEntities: [cluster]</code>	All in-cluster pod and reserved identities (broad)
<code>toEntities: [world]</code>	External (non-cluster) destinations only
<code>toCIDR: ["10.10.103.0/24"]</code>	A specific external CIDR
<code>toServices: [...]</code>	A specific Kubernetes Service (Cilium L4)
K8s <code>ipBlock: {cidr: ...}</code>	Only the <code>cidr</code> reserved identity — does NOT match host/remote-node/world identities. Frequent footgun.

K8s NetworkPolicy `ipBlock` does NOT match Cilium's reserved identities. If you need to allow egress to a node IP, use a CiliumNetworkPolicy with `toEntities`, not a K8s NetworkPolicy with `ipBlock`.

5. Is there a default-deny baseline already in scope?

```
# Cluster-wide
kubectl get ccnp
# Per-namespace
kubectl get networkpolicy,cnp -n <namespace>
```

After the 2026-05-21 refactor, there is **no cluster-wide default-deny**. The 2 platform CCNPs are additive. Per-namespace default-deny only exists where an app team has explicitly added one.

If you're adding a policy that selects all pods in a namespace and has `enableDefaultDeny: true`, **every pod in that namespace becomes default-deny** for that direction. Make sure that's intended.

6. Did you test in a non-prod namespace first?

For anything cluster-wide (CCNP, CiliumClusterwideNetworkPolicy, Kyverno ClusterPolicy):

1. Apply with `validationFailureAction: Audit` (Kyverno) or scoped to one test namespace (Cilium).
2. Watch for 24h with `hubble observe --verdict DROPPED` and Kyverno PolicyReport.
3. Promote to Enforce / cluster-wide only after the audit window is clean.

7. Have you noted the runbook impact?

If the policy could trigger a 4xx response or a connection drop in normal operation, **document the error message** in `runbook-dealing.md` under IR-6. Future on-call should be able to grep the message and find the policy.

8. (Kyverno specifically) Audit vs Enforce vs Mutate?

Mode	When
<code>Audit</code>	Always start here. Kyverno logs violations in <code>PolicyReport</code> / <code>ClusterPolicyReport</code> . No request is blocked. Watch reports for at least a few days.
<code>Enforce</code>	Flip once you've confirmed audit-mode reports are clean. Now violations get rejected at admission time.
<code>Mutate</code>	Different rule type — Kyverno modifies the resource (adds labels, sets defaults). Use sparingly; mutations are surprising.

Switching directly from "doesn't exist" to `Enforce` is how you accidentally block legit traffic.

After you `kubectl apply`

```
# 1. NetworkPolicy / CNP / CCNP — confirm it's accepted
kubectl get cnp,ccnp,netpol -A | grep <name>
kubectl describe <kind> <name> | sed -n '/Status:/,$p' # look for "Valid"

# 2. Watch drops for 5-10 min
W1_AGENT=$(kubectl -n kube-system get pod -l k8s-app=cilium --field-selector spec.nodeName=dealing-w-1 -o jsonpath='{.items[0].metadata.name}')
kubectl -n kube-system exec $W1_AGENT -c cilium-agent -- hubble observe --verdict DROPPED --last 50

# 3. Kyverno-side: check PolicyReports for new violations
kubectl get clusterpolicyreport -o json | python3 -c "
import sys, json
for r in json.load(sys.stdin)['items']:
    for res in r.get('results',[]):
        if res.get('result') != 'pass':
            print(res.get('policy'), '→', res.get('result'), ':', res.get('message',''))"
```

When to escalate

If a NetworkPolicy or CCNP you applied causes any of these — revert immediately and discuss:

- Ingress UIs return 403 / 404 (you probably caught the `ingress` reserved identity)
- Pods can no longer reach DNS (`coredns` returns nothing → blocked egress on port 53)
- Pods can no longer reach kube-apiserver (kubectl from pods fails)
- Argo Application sync starts failing across the board
- Prometheus targets all go `up=0` at once

Revert: `kubectl delete -f <file>` (and remove the live resource if not in IaC).

Links

- [Cilium NetworkPolicy concepts](#)
- [Kyverno docs](#)
- `runbook-dealing.md` — what to do when policy issues hit production
- `workload-catalog-dealing.md` — what's running where
- Existing policies: `security/network-policies/`, `security/kyverno-policies/`